

# Módulo V: Taller Integrativo

## Clase 3 — Reportería en Python: cálculo y visualización

2026-06-30

### Contacto

-  sebaegana@gmail.com
  -  <https://segana.netlify.app>
  -  <https://www.linkedin.com/in/sebastian-egana-santibanez/>
  -  <https://github.com/sebaegana>
- 

### Módulo V

#### Clase 3: Reportería en Python

##### Cálculo y visualización de KPIs con Pandas y Plotly

---

### Objetivo de la sesión

- Cargar y limpiar los datos de la organización con Pandas
  - Calcular los KPIs definidos en la Clase 2
  - Visualizar indicadores con Plotly
  - Estructurar un DataFrame de resultados consolidado
-

## Flujo de trabajo

Datos fuente (Excel / CSV)  
↓  
Carga con Pandas  
↓  
Limpieza y transformación  
↓  
Cálculo de KPIs  
↓  
Visualización (Plotly)  
↓  
Exportación de resultados  
↓  
Reporte formal ← Clase 4

---

## Carga de datos

```
import pandas as pd

# Desde Excel
df = pd.read_excel("datos_organizacion.xlsx", sheet_name="ventas")

# Desde CSV
df = pd.read_csv("datos_organizacion.csv", encoding="utf-8")

# Vista rápida
df.head()
df.info()
df.describe()
```

---

## Limpieza básica

```
# Eliminar filas con valores nulos en columnas clave
df = df.dropna(subset=["ventas", "periodo"])

# Convertir fechas
df["periodo"] = pd.to_datetime(df["periodo"])

# Renombrar columnas
df = df.rename(columns={"Ventas Netas": "ventas", "Costos Op.": "costos"})

# Filtrar período relevante
df = df[df["periodo"].dt.year == 2024]
```

---

## Cálculo de KPIs

```
# Margen operacional
df["margen"] = (df["ventas"] - df["costos"]) / df["ventas"] * 100

# Tasa de conversión
df["tasa_conv"] = df["ventas_cerradas"] / df["contactos"] * 100

# Agrupado por mes
resumen = df.groupby(df["periodo"].dt.month).agg(
    ventas=("ventas", "sum"),
    costos=("costos", "sum")
).assign(margen=lambda x: (x["ventas"] - x["costos"]) / x["ventas"] * 100)
```

---

## Visualización con Plotly

```
import plotly.express as px

# Gráfico de línea con meta
fig = px.line(
    resumen.reset_index(),
    x="periodo",
    y="margen",
    title="Margen Operacional Mensual",
    labels={"margen": "Margen (%)", "periodo": "Mes"}
)
fig.add_hline(y=20, line_dash="dash",
              annotation_text="Meta: 20%",
              line_color="red")
fig.show()
```

---

## Visualización de múltiples KPIs

```
import plotly.graph_objects as go
from plotly.subplots import make_subplots

fig = make_subplots(rows=2, cols=2,
                    subplot_titles=["Margen", "Conversión", "Retención", "NPS"])
```

```
fig.add_trace(go.Scatter(x=df["mes"], y=df["margen"]), row=1, col=1)
fig.add_trace(go.Bar(x=df["mes"], y=df["tasa_conv"]), row=1, col=2)
fig.add_trace(go.Scatter(x=df["mes"], y=df["retencion"]), row=2, col=1)
fig.add_trace(go.Bar(x=df["mes"], y=df["nps"]), row=2, col=2)

fig.update_layout(title="Dashboard de KPIs", showlegend=False)
fig.show()
```

---

## Exportar resultados

```
# Exportar DataFrame de KPIs
resumen.to_excel("kpis_calculados.xlsx", index=True)
resumen.to_csv("kpis_calculados.csv", index=True)

# Exportar gráfico como imagen
fig.write_image("dashboard_kpis.png")
fig.write_html("dashboard_kpis.html")
```

---

## Trabajo de la sesión

1. Cargar los datos de la organización en Python
  2. Limpiar y preparar el DataFrame
  3. Calcular los KPIs definidos en la Clase 2
  4. Generar al menos un gráfico por indicador
  5. Exportar el DataFrame de resultados a Excel
- 

## Próxima clase

### Clase 4 — Reportería en Python: generación de reportes Excel

- Estructura de un reporte de control de gestión
- Formato profesional con openpyxl
- Semáforo de cumplimiento de metas
- Integración de gráficos en el reporte